

1. Be the function

Name: _____ Partner: _____

Work on paper first. One partner is the caller; the other is the function. Pass the input and returned value on separate slips. Swap roles for the second call.

```
int change(int n) {
    n = n * 2;
    return n + 1;
}
```

Round A: Store the result in a different variable

```
int number = 4;
int result = change(number);
```

Draw separate memory boxes for number, n, and result. Show n at entry and after multiplication. Mark when the function call ends; n is local to that call.

Final number: _____ Final result: _____ Returned value: _____

Round B: Assign the result back to the caller

```
int number = 4;
number = change(number);
```

Start again from 4. Trace the call. Which statement changes number?

Explain why assigning to n does not itself change number.

2. Predict, run, explain

Predict individually. Compare with a partner before running the complete sketch.

```
int addOne(int x) {
    return x + 1;
}

void setup() {
    int a = 5;
    addOne(a);
    println(a);
    a = addOne(a);
    println(a);
    println(addOne(a));
    println(a);
}
```

Record the four printed values in order. Beside each, explain why that value appears.

1. Value: _____ Reason: _____

2. Value: _____ Reason: _____

3. Value: _____ Reason: _____

4. Value: _____ Reason: _____

What happens to the value returned by the first call, `addOne(a)`; ?

Change one statement so the final value of `a` is 7. Write the replacement and explain it.

3. Function repair shop

For each: identify the problem, write a repair, and give a test with its expected result.

A. Use the supplied input

```
float twice(float value) {
    return mouseX * 2;
}
```

The intended result is twice the supplied value. Does this compile? Does it meet the rule?

B. Return a calculation

```
int square(int n) {
    println(n * n);
}
```

The caller needs the square as a returned integer. Explain why printing is insufficient.

C. Cover every return path

```
boolean isPositive(int n) {
    if (n > 0) {
        return true;
    }
}
```

Repair the function. Test a negative number, zero, and a positive number.

4. Plan, test, explain

Use this page for open studio. Choose your own subject, interaction, and rule.

My idea

What will the viewer see or do? What repeats, and what changes?

My functions

Plan a drawing function and a value-returning function. For each, write its header, responsibility, input meanings, and result (or drawing effect).

A test and a partner check

Choose a boundary case. What input will you use, and what should happen?

Trace one value-returning call with your partner. Where do its inputs come from? How does the caller use the returned value?

Exit question

How does changing the screen differ from returning a value? Refer to your own sketch, or to the lecture examples.
